

Script — Demo API REST | Laboratorio PF3325 (~6 min)

Presentadores: Gabriel Fallas & Valeria Chinchilla

Material: presentacionlab.html (9 slides) + API corriendo en localhost:8000

Antes de grabar

1. Arrancar la API:

```
uvicorn src.api_phishing:app --reload
```

2. Abrir presentacionlab.html en el navegador (F11 pantalla completa)

3. Tener listas otras dos pestañas: localhost:8000 y localhost:8000/docs

Reparto

- **Gabriel:** slides 1–4 (contexto, arquitectura, endpoints)
- **Valeria:** slides 5–9 (código, extractor, demo, conclusión)

Guion

Slide 1 — Portada · (~15 s) — *Valeria*

"Hola, somos Gabriel y Valeria. Vamos a mostrar la implementación de nuestra API REST de detección de phishing en tiempo real."

Slide 2 — Contexto · (~45 s) — *Valeria*

"Durante el semestre entrenamos un MLP para clasificar sitios de phishing. El modelo quedó con 96.5% de accuracy y 98.8% de recall — pero guardado en disco no le sirve a nadie."

Lo que queríamos era que alguien pueda mandar una URL cualquiera y recibir un veredicto sin necesitar Python ni el código. Para eso construimos esta API REST."

Slide 3 — Arquitectura · (~50 s) — Valeria

"El sistema tiene tres partes que trabajan en cadena. Primero el extractor de features: toma la URL y la convierte en los 30 valores que el modelo necesita. Segundo, el modelo y el scaler que están guardados en disco y se cargan al arrancar. Y tercero, la API con FastAPI que orquesta todo: recibe la solicitud, llama al extractor y al modelo, y devuelve el resultado.

Para arrancarlo es un solo comando: `uvicorn src.api_phishing:app --reload .`"

Slide 4 — Endpoints · (~45 s) — Valeria

"La API tiene cuatro endpoints. El GET en `/` es la demo web para probar desde el navegador. El GET `/health` dice el estado del servicio. El POST `/predict/features` recibe los 30 números directamente. Y el POST `/predict/url` es el principal: recibe una URL cruda, hace todo el trabajo solo y devuelve el veredicto.

(Transición a Valeria.)"

Slide 5 — Código: carga del modelo · (~50 s) — Gabriel

"A nivel de código, lo primero es cómo cargamos el modelo. Usamos carga lazy: el modelo y el scaler se cargan la primera vez que alguien hace una predicción, no al arrancar el servidor. Esto hace que el startup sea instantáneo.

La función de predicción toma el vector, lo normaliza con el scaler — usando la misma transformación que se ajustó durante el entrenamiento — y lo pasa por el MLP. Devuelve las probabilidades para ambas clases y calcula la confianza como la probabilidad de la clase predicha."

Slide 6 — Código: endpoint principal · (~55 s) — *Gabriel*

"El endpoint más importante es `/predict/url`. Recibe la URL y el timeout, llama al extractor, y el extractor devuelve tres cosas: el vector de 30 features, la procedencia de cada feature — si se midió en vivo o es un valor por defecto — y metadata sobre la URL.

Con eso llama a la función de predicción y arma la respuesta JSON que ven a la derecha. Incluye el veredicto, la confianza, la metadata de la URL, y por cada feature su valor y si se midió o no. Esto permite que quien consuma la API entienda con cuánta evidencia se tomó la decisión."

Slide 7 — Extractor · (~45 s) — *Gabriel*

"El extractor mide 24 de los 30 features en vivo. Parsea la URL, hace un handshake TLS real para ver el certificado, cuenta los redirects, baja el HTML con BeautifulSoup para buscar anchors externos, formularios e iframes, y consulta DNS y WHOIS para la antigüedad del dominio.

Los otros 6 features usan un valor neutro porque los servicios de donde venían ya no existen — Alexa cerró en 2022, el PageRank público en 2016. En vez de esconderlo, lo marcamos explícitamente como 'default' en la respuesta."

Slide 8 — Demo · (~1:00) — *Gabriel*

"Ahora lo vemos funcionando.

[Cambiar a localhost:8000]

Primero la demo web. Pruebo con `github.com` ... vemos que tarda menos de un segundo y sale Legítimo con alta confianza. La tabla muestra los 30 features, cuáles se midieron en vivo y cuáles son default, y el contador de 24/30.

Ahora pruebo con la URL sospechosa de ejemplo... sale Phishing.

[Cambiar a localhost:8000/docs]

FastAPI genera automáticamente la documentación Swagger. Puedo llamar `/predict/url` directamente desde acá. Y si prefieren desde la terminal:

```
curl -s -X POST localhost:8000/predict/url \  
-H 'Content-Type: application/json' \  
-d '{"url":"https://www.github.com}"'
```

Slide 9 — Conclusión · (~20 s) — *Gabriel*

"En resumen, la API recibe una URL, extrae los features en vivo, corre el modelo y devuelve el veredicto con confianza y procedencia de cada feature, todo en menos de un segundo. Gracias."